

CondoHub

Plataforma mobile de governança residencial voltada à transparência, organização e participação dos moradores na gestão condominial.

PROJETO ESG · PILAR PRINCIPAL: GOVERNANÇA

Aplicativo Android nativo desenvolvido em Kotlin com Jetpack Compose, composto por 11 telas e integrado a uma API pública de qualidade do ar.

ALUNO

Breno Laurentino

CURSO / TURMA

DISCIPLINA

PROFESSOR

DATA DE ENTREGA

REPOSITÓRIO

<https://github.com/Juubb/Condohub>

1. Objetivo do aplicativo

O **CondoHub** é uma plataforma mobile de governança residencial. Seu objetivo é centralizar, em um único aplicativo, as informações e os processos de decisão de um condomínio, dando ao morador acesso direto ao que hoje circula de forma dispersa.

O problema

Informações de condomínios costumam ficar espalhadas entre grupos de WhatsApp, murais do hall, comunicados impressos e o caderno da portaria. Essa dispersão gera três efeitos:

- o morador não encontra a informação quando precisa dela;
- decisões coletivas dependem de presença física em assembleia, o que reduz a participação;
- pedidos e reclamações não deixam rastro, o que dificulta a cobrança e a prestação de contas.

A solução

O aplicativo reúne notícias, eventos, regras, reservas de espaços, manutenções, informações de garagem e processos de participação em uma única plataforma. Cada solicitação gera um **protocolo**, e as decisões coletivas passam a acontecer por **votação registrada no aplicativo**, com resultado parcial visível a todos.

Resumo do valor entregue: o morador deixa de depender de terceiros para saber o que acontece no condomínio, e a administração passa a ter um registro rastreável de solicitações e decisões.

2. Tecnologia escolhida

O projeto foi desenvolvido como **aplicativo Android nativo**, utilizando a stack oficial recomendada pelo Google para desenvolvimento moderno na plataforma.

ITEM	ESCOLHA	MOTIVO
Plataforma	Android nativo	Equipamento disponível para desenvolvimento e teste é Windows + Android
Linguagem	Kotlin 2.0.21	Linguagem oficial do Android, com null safety e corrotinas nativas
Interface	Jetpack Compose	Toolkit declarativo atual; menos código que XML e recomposição automática
Design	Material 3	Componentes prontos e acessíveis, com identidade visual customizada
Navegação	Navigation Compose 2.8.4	Pilha de telas com passagem de parâmetros e botão voltar do sistema
Rede	HttpURLConnection + org.json	Ambos nativos do Android: evita dependências externas e reduz o tamanho do APK
Assincronia	Kotlin Coroutines	Consulta à API roda em Dispatchers.IO, sem travar a interface
Persistência	Estado em memória	MVP sem back-end, conforme o enunciado permite
SDK mínimo	API 24 (Android 7.0)	Cobre praticamente toda a base de aparelhos em uso
SDK alvo	API 35 (Android 15)	Versão atual exigida pela Play Store

Arquitetura

O código está organizado por responsabilidade, o que facilita localizar e evoluir cada parte:

```

app/src/main/java/com/example/condohub/
├─ MainActivity.kt          Ponto de entrada + NavHost com as 11 rotas
├─ navegacao/
│   └─ Navegacao.kt       Constantes de rota e titulos da barra superior
├─ modelo/
│   └─ Modelos.kt          Classes de dados (Evento, Pauta, Reserva, ...)
├─ dados/
│   ├── Repositorio.kt     Fonte de dados em memoria, observavel pelo Compose
│   └─ QualidadeArApi.kt   Consumo da API externa de qualidade do ar
└─ ui/
    ├── theme/              Cores, tipografia e tema do app
    ├── componentes/        Pecas reutilizadas (barra, selos, listas)
    └─ telas/               As 11 telas do aplicativo

```

O `Repositorio` usa `mutableStateListOf`, o que faz com que qualquer alteração feita pelo usuário — votar, reservar, criar evento — redesenhe automaticamente as telas que dependem daquele dado, sem código extra de atualização.

3. Aplicação no contexto ESG

O CondoHub atua nos três pilares do ESG, com foco principal em **Governança**. O condomínio é, na prática, uma organização com orçamento, corpo eleito, assembleia e prestação de contas — o que torna o pilar G o mais natural e o mais demonstrável.

G

PILAR PRINCIPAL

Governança

Torna transparentes e rastreáveis os processos de decisão e de cobrança:

- **Assembleia digital:** votação por pauta, com resultado parcial em tempo real e possibilidade de alterar o voto até o encerramento — substitui a votação presencial de braços levantados;
- **Ocorrências com protocolo:** cada registro recebe um número (OC-2026-0001), dando comprovante ao morador e histórico à administração;
- **Regimento interno acessível:** as regras deixam de ficar apenas na portaria;
- **Corpo de eleitos identificado:** quem responde pelo condomínio, com cargo, unidade e mandato vigente;
- **Reservas registradas:** substituem o caderno da portaria, com protocolo e verificação de conflito.

S

PILAR SECUNDÁRIO

Social

Melhora a comunicação e a convivência entre moradores, ampliando a participação na vida coletiva:

- agenda unificada de eventos, com confirmação de presença que ajuda a dimensionar os espaços;
- **qualquer morador pode propor um evento**, e não apenas a síndica — a proposta vai direto ao mural;
- feira de trocas e atividades coletivas incentivam consumo consciente e vínculo entre vizinhos.

E

PILAR SECUNDÁRIO

Ambiental

Implementado pela tela de coleta sustentável, que combina orientação local com **dado ambiental real consumido de uma API pública**:

- calendário semanal de coleta seletiva e guia de separação por lixeira, que reduzem a contaminação de recicláveis;
 - **monitoramento da qualidade do ar** da região do condomínio, com índice europeu (EAQI), material particulado PM2.5 e PM10 e recomendação sobre o uso das áreas externas;
 - pauta em votação sobre placas solares, ligando o pilar ambiental ao de governança.
-

4. Endereço do serviço consumido

O aplicativo consome a **Open-Meteo Air Quality API**, serviço público e gratuito de dados ambientais, que não exige cadastro nem chave de acesso.

ENDEREÇO BASE

```
https://air-quality-api.open-meteo.com/v1/air-quality
```

REQUISIÇÃO FEITA PELO APLICATIVO

```
https://air-quality-api.open-meteo.com/v1/air-quality?latitude=-23.5505&longitude=-46.6333&current=europe  
an_aqi,pm10,pm2_5&timezone=America%2FSao_Paulo
```

RESPOSTA (TRECHO)

```
{  
  "latitude": -23.55,  
  "longitude": -46.633,  
  "current": {  
    "time": "2026-09-01T14:00",  
    "european_aqi": 24,  
    "pm10": 15.2,  
    "pm2_5": 9.4  
  }  
}
```

ASPECTO	DETALHE
Documentação	https://open-meteo.com/en/docs/air-quality-api
Autenticação	Não exige chave de API nem cadastro
Protocolo	HTTPS, resposta em JSON
Implementação	dados/QualidadeArApi.kt
Permissão	android.permission.INTERNET declarada no Manifest
Tratamento de erro	Tela exibe carregamento, mensagem de falha e botão "Tentar de novo"

Os valores recebidos são classificados segundo a escala europeia de qualidade do ar (0–20 boa, 21–40 razoável, 41–60 moderada, 61–80 ruim, acima de 80 muito ruim) e convertidos em uma recomendação prática para o uso das áreas comuns.

5. Descrição e funcionalidades das telas

O aplicativo possui **11 telas**, acima do mínimo de cinco exigido. Cada tela abaixo traz a captura da interface, suas funcionalidades e um comentário sobre a implementação.

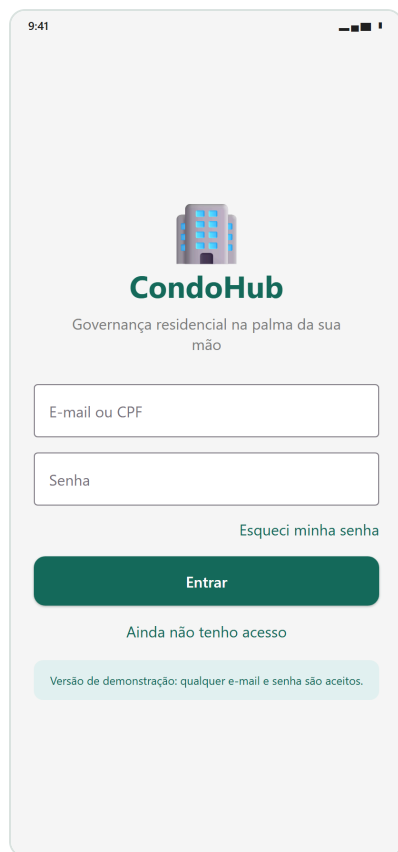


Figura 1 – Tela Login

TELA 1 DE 11

Login

ui/telas/TelaLogin.kt

Porta de entrada do morador. Apresenta a identidade do aplicativo e valida os dados de acesso antes de liberar o painel.

FUNCIONALIDADES

- Campos de e-mail/CPF e senha, com a senha mascarada
- Validação local: campo vazio fica vermelho e exibe a mensagem do erro
- Mensagem de retorno na parte inferior (Snackbar) quando faltam dados
- Links para recuperação de senha e solicitação de acesso

Como o MVP não possui back-end, a autenticação é simulada: qualquer par de credenciais preenchido libera o acesso. O estado dos campos usa `rememberSaveable`, então o texto digitado sobrevive à rotação da tela.



Figura 2 – Tela Home

TELA 2 DE 11

Home

ui/telas/TelaHome.kt

Painel principal. Reúne em uma só tela o que hoje fica espalhado em grupos de mensagem, murais e comunicados impressos.

FUNCIONALIDADES

- Cartão de identificação do condomínio e da unidade do morador
- Carrossel horizontal de eventos, arrastável com o dedo
- Grade com os serviços do condomínio, cada um com contador de registros
- Lista de avisos e manutenções programadas
- Botão de sair, que limpa os dados da sessão

O carrossel usa `LazyRow`, que já é arrastável por padrão no Android e só monta os cartões visíveis. A grade é montada com `chunked(2)`, mantendo duas colunas mesmo se novos serviços forem acrescentados.

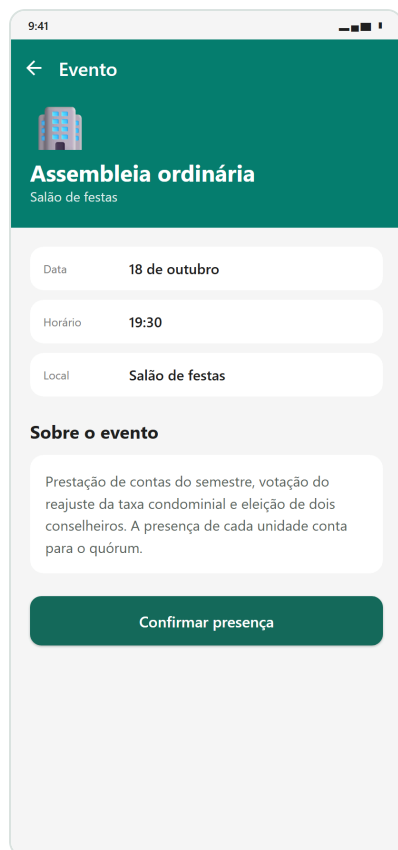


Figura 3 – Tela Detalhe do evento

TELA 3 DE 11

Detalhe do evento

ui/telas/TelaEvento.kt

Abre ao tocar em um cartão do carrossel. Traz a descrição completa da atividade e permite confirmar presença.

FUNCIONALIDADES

- Faixa de destaque com ícone, título e local do evento
- Data, horário e local em cartões separados
- Descrição completa da atividade
- Botão que alterna entre confirmar e cancelar presença
- Opção de excluir, só para eventos criados pelo próprio morador

A confirmação de presença ajuda a administração a dimensionar o espaço, o que caracteriza o **pilar Social**. O identificador do evento chega pela rota `evento/{eventoId}` e a presença é lida de um `mutableStateMap`, refletindo na Home na mesma hora.



Figura 4 – Tela Novo evento

TELA 4 DE 11

Novo evento

ui/telas/TelaNovoEvento.kt

Formulário em que qualquer morador propõe uma atividade para o mural do condomínio, ampliando a participação da comunidade.

FUNCIONALIDADES

- Seletor de ícone com 12 opções
- Campo de título com validação de obrigatoriedade
- Calendário nativo do Material 3 para escolher a data
- Chips de horário e campo de local
- Descrição opcional, com texto padrão se ficar vazia

O evento criado entra na lista já **ordenado por data** junto com os demais e recebe marcação de autoria, o que permite ao morador excluí-lo depois. A data escolhida chega em milissegundos UTC e é convertida com `Calendar` para evitar deslocamento de fuso.



Figura 5 – Tela Votações

TELA 5 DE 11

Votações

ui/telas/TelaVotacoes.kt

Assembleia digital: núcleo do pilar de Governança. Cada morador vota nas pautas em aberto e acompanha o resultado parcial.

FUNCIONALIDADES

- Resumo de quantas pautas o morador já votou
- Cartão por pauta, com a proposta e o prazo de encerramento
- Barra proporcional de votos a favor e contra, com percentual e total
- Botões A favor / Contra, destacados conforme o voto do morador
- Selo indicando as pautas já votadas

É a tela mais representativa do **pilar G**: substitui a votação presencial por um processo rastreável e visível a todos. A regra desfaz o voto anterior antes de somar o novo, então trocar de opinião não infla o total — comportamento coberto por teste unitário.



Figura 6 – Tela Reservar espaço

TELA 6 DE 11

Reservar espaço

ui/telas/TelaReserva.kt

Substitui o caderno da portaria por um registro com protocolo, visível ao morador e à administração.

FUNCIONALIDADES

- Escolha entre salão de festas, churrasqueira, quadra e espaço gourmet
- Calendário para a data e chips para o turno
- Verificação que impede reserva duplicada no mesmo espaço, data e horário
- Lista das reservas do morador com protocolo e status
- Cancelamento de reserva já solicitada

Toda reserva nasce com status **Pendente**, refletindo o fluxo real de aprovação pela síndica. A verificação de conflito também é coberta por teste unitário.

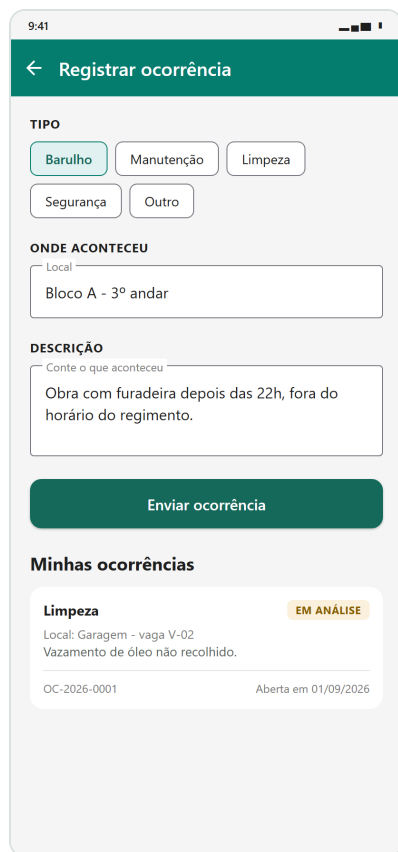


Figura 7 – Tela Registrar ocorrência

TELA 7 DE 11

Registrar ocorrência

ui/telas/TelaOcorrencia.kt

Canal formal para reclamações e pedidos de manutenção, com número de protocolo e histórico.

FUNCIONALIDADES

- Classificação por tipo: barulho, manutenção, limpeza, segurança ou outro
- Campos de local e descrição, ambos obrigatórios
- Geração automática de protocolo no formato OC-2026-0001
- Histórico das ocorrências abertas, com data e status
- Limpeza automática do formulário após o envio

É a face fiscalizatória do **pilar G**: o protocolo dá ao morador um comprovante e à administração um histórico auditável, encerrando o problema do pedido feito verbalmente que ninguém registra.

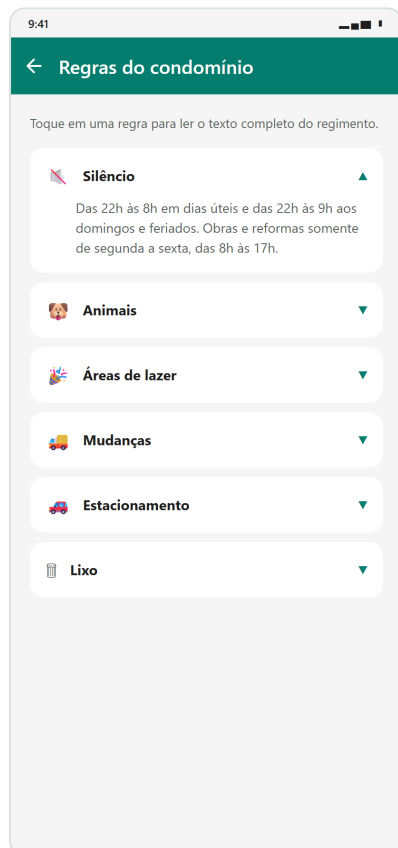


Figura 8 – Tela Regras do condomínio

TELA 8 DE 11

Regras do condomínio

ui/telas/TelaRegras.kt

Regimento interno acessível a qualquer momento, no lugar do documento impresso que fica só na portaria.

FUNCIONALIDADES

- Sete regras: silêncio, animais, áreas de lazer, mudanças, estacionamento, lixo e visitantes
- Cada item expande ao toque, revelando o texto completo
- Ícone indicando se o item está aberto ou fechado

A expansão usa `animateContentSize`, que anima a mudança de altura sem código adicional. Transparência de regras é um item clássico de governança: ninguém pode ser cobrado por uma norma que não conseguia consultar.

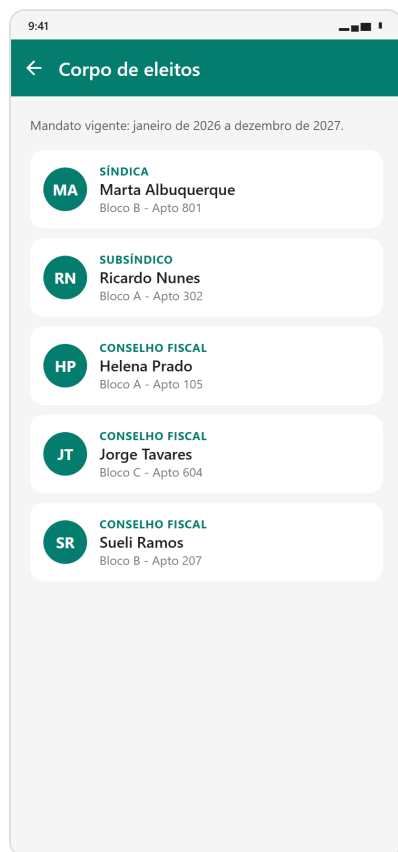


Figura 9 – Tela Corpo de eleitos

TELA 9 DE 11

Corpo de eleitos

ui/telas/TelaEleitos.kt

Mostra quem responde pelo condomínio e onde encontrar cada um dos eleitos.

FUNCIONALIDADES

- Síndica, subsíndico e três membros do conselho fiscal
- Avatar gerado a partir das iniciais do nome
- Cargo, nome e unidade de cada eleito
- Indicação do mandato vigente

Saber quem está no comando é a base da prestação de contas. As iniciais do avatar são calculadas em tempo de execução a partir do nome, dispensando imagens no APK e mantendo o pacote pequeno.

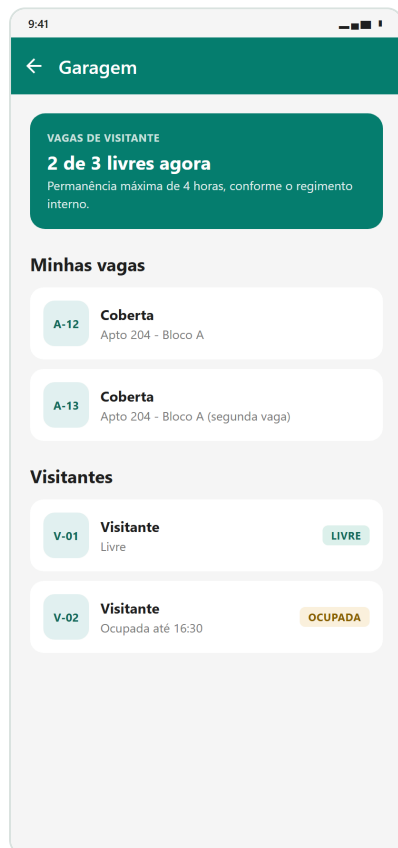


Figura 10 – Tela Garagem

TELA 10 DE 11

Garagem

ui/telas/TelaGaragem.kt

Mapa das vagas do condomínio, separadas por finalidade.

FUNCIONALIDADES

- Contador de vagas de visitante livres no momento
- Vagas da unidade do morador
- Vagas rotativas de visitante, com selo de livre ou ocupada
- Vagas de uso especial: carga de veículo elétrico e acessibilidade

As vagas de carga elétrica reforçam o **pilar ambiental** na infraestrutura do condomínio. O contador de vagas livres é calculado a partir da lista, então acompanha qualquer alteração nos dados.

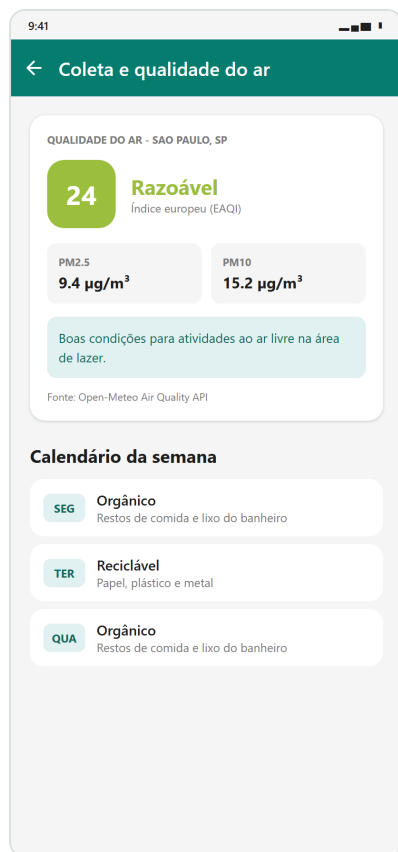


Figura 11 – Tela Coleta e qualidade do ar

TELA 11 DE 11

Coleta e qualidade do ar

ui/telas/TelaColeta.kt

Pilar ambiental e única tela que consome serviço externo. Combina orientação local de reciclagem com dado ambiental real.

FUNCIONALIDADES

- Índice europeu de qualidade do ar (EAQI), com cor conforme a faixa
- Concentrações de material particulado PM2.5 e PM10 em $\mu\text{g}/\text{m}^3$
- Recomendação prática sobre o uso das áreas externas
- Calendário semanal de coleta seletiva e guia de separação por lixeira
- Estados de carregamento e de erro, com botão para nova tentativa

A consulta roda em `Dispatchers.IO` dentro de um `LaunchedEffect`, mantendo a interface responsiva. O estado da tela é um `sealed interface` com três casos — Carregando, Sucesso e Falha — garantindo que nenhuma situação fique sem tratamento visual.

6. Como executar e gerar o APK

1 Abrir o projeto

No Android Studio: *File* → *Open* e selecionar a pasta raiz do projeto. Aguardar a sincronização do Gradle, que baixa as dependências na primeira execução.

2 Executar

Selecionar um emulador ou aparelho físico com depuração USB ativada e clicar em *Run*. A tela de login aceita qualquer e-mail e senha preenchidos.

3 Limpar o projeto

Build → *Clean Project*, para que o pacote final não carregue artefatos de compilações anteriores.

4 Gerar o APK assinado

Build → *Generate Signed App Bundle / APK* → *APK*. Criar um keystore novo, guardar a senha, selecionar a variante **release** e concluir.

5 Localizar o arquivo

O APK fica em `app/release/app-release.apk`. Ele deve ser colocado **fora** da pasta do projeto dentro do ZIP de entrega.

Antes de enviar: confirme que o ZIP contém as três partes exigidas — a pasta do projeto compactada (com todo o código-fonte), o APK em modo release separado da pasta do projeto, e este relatório em PDF.